

Neural Network Basics

Neural networks are built from simple units stacked in layers. This module explains a single unit, the functions that make networks non-linear, and the calculus rule that makes training them possible.

Perceptron

Prerequisites: Linear Regression.

The perceptron, introduced by Frank Rosenblatt in 1958, is the simplest neural unit. It computes a weighted sum of its inputs plus a bias, exactly like linear regression, and then applies a step function: output 1 if the sum is above zero, otherwise 0.

The perceptron learning rule adjusts the weights whenever a training example is misclassified, nudging the decision boundary toward the correct side. If the two classes can be separated by a straight line (they are linearly separable), the perceptron is guaranteed to find a separating line.

Its famous limitation is that a single perceptron cannot learn the XOR function, because XOR is not linearly separable. This result slowed neural network research for years.

The fix is to stack units into layers. A multi-layer perceptron (MLP) has an input layer, one or more hidden layers and an output layer. With a non-linear activation function in the hidden layers, an MLP can learn XOR and, in principle, approximate any continuous function.

Activation Functions

Prerequisites: Perceptron.

An activation function is applied to each neuron's weighted sum. Without non-linear activations, stacking layers is pointless: a chain of linear layers collapses into a single linear layer.

The sigmoid function maps any number to the range 0 to 1 and was popular early on, but for large positive or negative inputs its slope is almost zero. This causes the vanishing gradient problem: gradients become tiny in early layers and those layers stop learning. Tanh maps to the range -1 to 1 and is zero-centred but has the same problem.

ReLU, the rectified linear unit, outputs $\max(0, x)$. It is cheap to compute and does not saturate for positive inputs, which is why it is the default choice for hidden layers today. Its weakness is "dying ReLU": a neuron stuck in the negative region outputs zero and stops learning. Leaky ReLU fixes this with a small slope for negative inputs.

For the output layer, use sigmoid for binary classification, softmax for multi-class classification (it turns scores into probabilities that sum to 1), and no activation for regression.

Chain Rule

Prerequisites: Derivatives.

The chain rule tells us how to differentiate a function composed of other functions. If y depends on u and u depends on x , then the derivative of y with respect to x equals the derivative of y with respect to u multiplied by the derivative of u with respect to x . In short: $dy/dx = dy/du \text{ times } du/dx$.

Example: for $y = (3x + 1)$ squared, let $u = 3x + 1$. Then $dy/du = 2u$ and $du/dx = 3$, so $dy/dx = 2(3x + 1) \text{ times } 3 = 6(3x + 1)$.

A neural network is one long composition of functions: each layer applies weights, then an activation, and passes the result to the next layer, and the loss sits at the end of the chain. To know how the loss changes when one weight deep inside the network changes, we multiply the local derivatives along the path from the loss back to that weight.

Applying the chain rule systematically from the output back to the input, reusing intermediate results, is exactly the backpropagation algorithm. If backpropagation feels confusing, the chain rule is almost always the missing piece.